

RAG ChatBot with History

Conversational RAG With PDF uploads and chat history

Upload Pdf's and chat with their content

Enter your Groq API key:

.....



Session ID

1

Choose A Pdf file



Drag and drop files here

Limit 200MB per file • PDF

Browse files



Limitless.pdf 3.8MB



Your question:

how can i remember 50 floors?

▶ { . . . }

Assistant: That's a lot of floors!

While the loci method is powerful, remembering 50 floors might be pushing it, even with diligent practice. Here are a few things to consider:

- **Break it down:** Could you group your 50 points into smaller categories? Instead of 50 floors, maybe you have 5 main categories, each with 10 floors. This makes the task more manageable.
- **Visualization:** The more vivid and unique your visualizations are, the easier they'll be to recall. Think about creating memorable scenes or stories at each location.
- **Spaced Repetition:** Regularly reviewing your "palace" and its contents will help solidify the memories.
- **Alternatives:** If 50 points is truly essential, explore other memory techniques alongside loci, like the peg system or creating memory chains.

Remember, the key is to find a system that works best for you and your learning style.

Chat History:

▶ [

0 :

"HumanMessage(content='Summarize the gist in a paragraph', additional_kwargs=

1. Import the libraries

```
import streamlit as st
from langchain.chains import create_history_aware_retriever, create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_chroma import Chroma
from langchain_community.chat_message_histories import ChatMessageHistory
from langchain_core.chat_history import BaseChatMessageHistory
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_groq import ChatGroq
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import PyPDFLoader
import os
```

2. Set environment variables

```
from dotenv import load_dotenv
load_dotenv()

os.environ['HF_TOKEN']=os.getenv("HF_TOKEN")
```

3. Embeddings

```
embeddings=HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
```

4. Set up Streamlit

```
## set up Streamlit
st.title("Conversational RAG With PDF uploads and chat history")
st.write("Upload Pdf's and chat with their content")
```

Enter API Key:

```
## Input the Groq API Key
api_key=st.text_input("Enter your Groq API key:",type="password")
```

5. Main Code

```
if uploaded_files:
    documents=[]
    for uploaded_file in uploaded_files:
        temppdf=f"./temp_{uploaded_file.name}"
        with open(temppdf,"wb") as file:
            file.write(uploaded_file.getvalue())
            file_name=uploaded_file.name

        loader=PyPDFLoader(temppdf)
        docs=loader.load()
        documents.extend(docs)
```

- **for uploaded_file in uploaded_files:**
 - Loops over each file in a collection of uploaded files.
- **temppdf = "./temp.pdf_{uploaded_file.name}"**
 - Sets the path to a temporary file called **temp.pdf_{uploaded_file.name}** in the current directory.

- `with open(temp_pdf, "wb") as file:`
 - Opens the file `temp.pdf` in **binary write mode** (`wb`).
 - The `with` statement ensures the file is closed properly after writing.
- `file.write(uploaded_file.getvalue())`
 - **Writes the contents of the uploaded file into the `temp.pdf` file.**
 - `uploaded_file.getvalue()` reads the **entire content** of the uploaded file (likely a `BytesIO` object).
- `file_name = uploaded_file.name`
 - Stores the original filename of the uploaded file for later use (e.g., saving, logging, or displaying).
 - *Example:* If someone uploads a file called `resume.pdf`, then: `uploaded_file.name` gives **"resume.pdf"**

So what's happening overall?

Imagine this:

1. A user uploads `report.pdf`.
2. Python reads it into memory as `uploaded_file`.
3. We want to **save it** to your machine — so we:
 - Open (create) a real file named `temp.pdf` using `"wb"`
 - Write all the data from the uploaded file into this new file
4. We also store the original file name (`report.pdf`) in `file_name` for reference.

6. Load PDF

```
loader=PyPDFLoader(temp_pdf)
docs=loader.load()
documents.extend(docs)
```

7. Split and create Embeddings

```
# Split and create embeddings for the documents
text_splitter = RecursiveCharacterTextSplitter(chunk_size=5000, chunk_overlap=200)
splits = text_splitter.split_documents(documents)
vectorstore = Chroma.from_documents(documents=splits, embedding=embedder)
retriever = vectorstore.as_retriever()
```

```
contextualize_q_system_prompt=(
    """
    Given a chat history and the latest user question which might reference
    context in the chat history, formulate a standalone question which can be understood
    without the chat history. Do NOT answer the question, just reformulate it if needed
    and otherwise return it as is.
    """
)
contextualize_q_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", contextualize_q_system_prompt),
        MessagesPlaceholder("chat_history"),
        ("human", "{input}"),
    ]
)
```

contextualize_q_system_prompt : This is a set of instructions for the AI model. It tells the model: "Given the conversation so far, and the latest question, rephrase the question so it can be understood on its own, without needing the chat history.

Don't answer it, just rephrase if necessary."

```
history_aware_retriever=create_history_aware_retriever(llm,retriever,contextualize_q_prompt)
```

- This combines **AI model (`llm`)**, **document search tool (`retriever`)**, and the rephrasing prompt.
- Its job is to take a user's question, potentially rephrase it using chat history, and then use the `retriever` to find relevant document chunks based on the rephrased question.

8. Answering the Question (Question-Answering Chain Prompt):

- This creates the prompt for generating the final answer. It includes:

```
system_prompt = (
    "You are an assistant for question-answering tasks. "
    "Use the following pieces of retrieved context to answer "
    "the question. If you don't know the answer, say that you "
    "don't know. Use three sentences maximum and keep the "
    "answer concise."
    "\n\n"
    "{context}"
)
qa_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system_prompt),
        MessagesPlaceholder("chat_history"),
        ("human", "{input}"),
    ]
)
```

`system_prompt` : This is another set of instructions for the AI model. It tells the model: "You are an assistant. Use the provided *context* (document chunks) to answer the question. If you don't know, say so. Keep it concise (max three sentences)."

```
question_answer_chain=create_stuff_documents_chain(llm,qa_prompt)
```

- This combines AI model (`llm`) with the question-answering prompt.

9. The Complete RAG Chain:

```
rag_chain=create_retrieval_chain(history_aware_retriever,question_answer_ch  
ain)
```

```
def get_session_history(session:str)→BaseChatMessageHistory:  
    if session_id not in st.session_state.store:  
        st.session_state.store[session_id]=ChatMessageHistory()  
    return st.session_state.store[session_id]
```

```
conversational_rag_chain=RunnableWithMessageHistory(  
    rag_chain,get_session_history,  
    input_messages_key="input",  
    history_messages_key="chat_history",  
    output_messages_key="answer"  
)
```

```
rag_chain=create_retrieval_chain(history_aware_retriever,question_answer_ch  
ain)
```

- It links the **history-aware retriever** (which gets relevant documents) with the **question-answer chain** (which generates the answer based on those documents).

Get session history (Std code):

```
def get_session_history(session:str)→BaseChatMessageHistory:  
    if session_id not in st.session_state.store:  
        st.session_state.store[session_id]=ChatMessageHistory()  
    return st.session_state.store[session_id]
```

```

conversational_rag_chain=RunnableWithMessageHistory(
    rag_chain,get_session_history,
    input_messages_key="input",
    history_messages_key="chat_history",
    output_messages_key="answer"
)

```

conversational_rag_chain=RunnableWithMessageHistory(...) :

This is the magic component that makes your RAG system conversational.

- It takes your `rag_chain` (the core RAG logic).
- It uses `get_session_history` to manage the chat history.
- `input_messages_key="input"` : Tells it where to find the user's current question.
- `history_messages_key="chat_history"` : Tells it where to inject the chat history into your prompts (matching `MessagesPlaceholder("chat_history")`).
- `output_messages_key="answer"` : Tells it where to find the AI's response.
- This `RunnableWithMessageHistory` automatically handles:
 - Adding the user's input to the history before processing.
 - Passing the full chat history to the `rag_chain` (where the `history_aware_retriever` and `question_answer_chain` use it).
 - Adding the AI's response to the history after processing.

10. User Interaction and Getting Answers (When you type a question)

```

user_input = st.text_input("Your question:")
if user_input:
    session_history=get_session_history(session_id)
    response = conversational_rag_chain.invoke(

```

```

        {"input": user_input},
        config={
            "configurable": {"session_id":session_id}
        }, # constructs a key "abc123" in `store`.
    )
    st.write(st.session_state.store)
    st.write("Assistant:", response['answer'])
    st.write("Chat History:", session_history.messages)
else:
    st.warning("Please enter the GRoq API Key")

```

User Input:

- `user_input = st.text_input("Your question:")` : This creates a text box where you can type your question

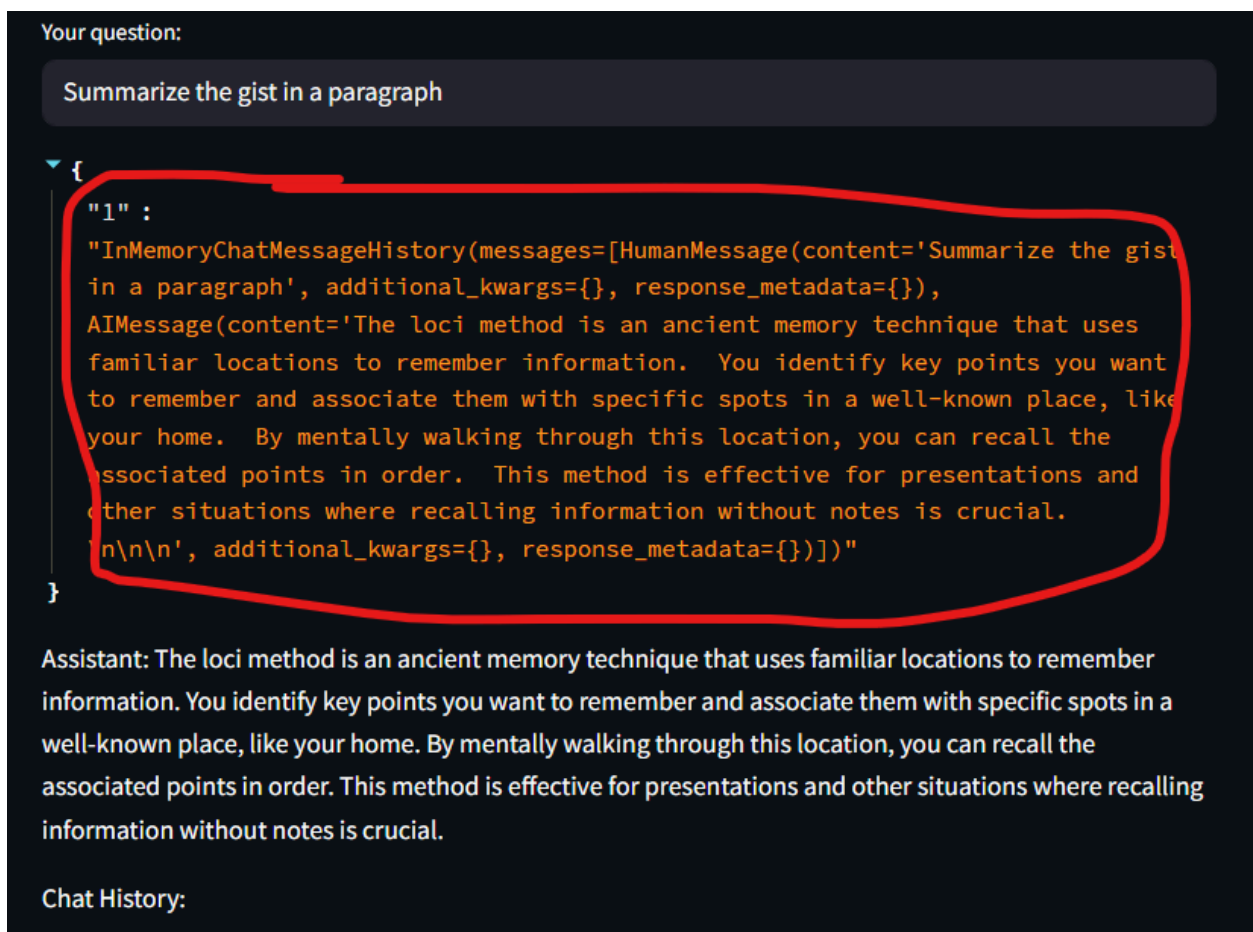
Processing the Question:

- `if user_input:` : This block runs when you type a question and press Enter.
- `session_history=get_session_history(session_id)` : It fetches the current chat history for your session.
- `response = conversational_rag_chain.invoke(...)` : This is where the whole system comes alive!
 - You give it the `user_input` .
 - `config={"configurable": {"session_id":session_id}}` : This tells `RunnableWithMessageHistory` which session's history to use.
 - The `conversational_rag_chain` then does the following in sequence:
 1. **Retrieves Chat History:** It gets the history for `session_id` .
 2. **Contextualizes Question:** It sends the `user_input` and `chat_history` to the `history_aware_retriever` . The AI rephrases the question if needed (e.g., "What was the capital of France?" instead of "What about that one?").
 3. **Retrieves Documents:** It uses the rephrased question (or original if no rephrasing was needed) to search your `vectorstore` and find the most relevant chunks of text from your uploaded PDFs.

4. **Generates Answer:** It then sends the *original user question*, the *full chat history*, and the *retrieved document chunks* to the `question_answer_chain`. The AI (`llm`) uses all this information to formulate a concise and relevant answer.
5. **Updates Chat History:** The `conversational_rag_chain` automatically adds your `user_input` and the AI's `response['answer']` to the `session_history`.

```
st.write(st.session_state.store)
```

- It displays the histories in `st.session_state.store`



The screenshot shows a Streamlit chat interface. At the top, it says "Your question:". Below that is a text input field containing "Summarize the gist in a paragraph". The AI response is displayed in a code block, which is highlighted with a red hand-drawn circle. The response is a JSON object containing a list of messages. Below the code block, the AI's response is rendered as a paragraph of text. At the bottom, it says "Chat History:".

```
{
  "1" :
  "InMemoryChatMessageHistory(messages=[HumanMessage(content='Summarize the gist in a paragraph', additional_kwargs={}, response_metadata={}),
  AIMessage(content='The loci method is an ancient memory technique that uses familiar locations to remember information. You identify key points you want to remember and associate them with specific spots in a well-known place, like your home. By mentally walking through this location, you can recall the associated points in order. This method is effective for presentations and other situations where recalling information without notes is crucial.'
  '\n\n', additional_kwargs={}, response_metadata={})])"
}
```

Assistant: The loci method is an ancient memory technique that uses familiar locations to remember information. You identify key points you want to remember and associate them with specific spots in a well-known place, like your home. By mentally walking through this location, you can recall the associated points in order. This method is effective for presentations and other situations where recalling information without notes is crucial.

Chat History:

`st.write("Chat History:", session_history.messages)` : Shows you the current state of your conversation history (for debugging/understanding).

Chat History:

```
▼ [
  0 :
  "HumanMessage(content='Summarize the gist in a paragraph', additional_kwargs=
  {}, response_metadata={})"
  1 :
  "AIMessage(content='The loci method is an ancient memory technique that uses
  familiar locations to remember information. You identify key points you want
  to remember and associate them with specific spots in a well-known place, like
  your home. By mentally walking through this location, you can recall the
  associated points in order. This method is effective for presentations and
  other situations where recalling information without notes is crucial.
  \n\n\n', additional_kwargs={}, response_metadata={})"
]
```

✨ **st.session_state.store** VS. **session_history.messages**

st.session_state.store :

The Master Memory Bank for All Sessions

session_history.messages :

The Actual Conversation Log for One Session

```
[
  HumanMessage(content="What is the capital of France?"),
  AIMessage(content="The capital of France is Paris."),
  HumanMessage(content="What is its population?"),
  # ... and so on
]
```

Why do we pass 2 prompts?

- **The first prompt (`contextualize_q_prompt`) is for the *retriever*:** It ensures the retriever receives a clear, complete, and non-ambiguous search query, leveraging the chat history to resolve references. This is about **finding the right information**.
- **The second prompt (`qa_prompt`) is for the *answer generation*:** It takes the *found information*, the *original user question*, and the *full chat history* to synthesize the final, natural-sounding, and contextually appropriate response. This is about **using the information to answer well**.



`create_stuff_documents_chain(llm, prompt)`

◆ Purpose:

Takes a list of documents, inserts them into a prompt (`{context}`), sends it to the LLM, and gets an answer.

◆ Used when:

You **already have the context**, and you want the LLM to generate a response.



`create_history_aware_retriever(llm, retriever, contextualize_prompt)`

◆ Purpose:

This **enhances the retriever** to handle **follow-up questions** by:

1. Looking at the **chat history** (previous user/AI messages)
2. Reformulating the current input into a **standalone question**
3. Then sending that to the **normal retriever**

◆ Used when:

You want the chatbot to support **conversational memory**, e.g.:

Q1: What is Nietzsche's philosophy?

Q2: What does he say about God?

In Q2, "he" needs to be resolved as "Nietzsche."

◆ Example:

```
contextualize_prompt = ChatPromptTemplate.from_messages([...])
history_aware_retriever = create_history_aware_retriever(
    llm, retriever, contextualize_prompt
)
```

So instead of:

```
retriever.invoke("What does he say about God?")
```

It becomes:

```
history_aware_retriever.invoke({
    "input": "What does he say about God?",
    "chat_history": [...]
})
```



create_retrieval_chain(retriever, document_chain)

◆ Purpose:

This is a **full RAG pipeline**:

- First: Uses the **retriever** to fetch relevant docs based on the input question.
- Then: Passes those docs into the **document_chain** (which uses LLM and prompt to answer).

◆ Used when:

You want to **automatically retrieve the right context** from a vector store before generating the answer.

◆ Example:

```
retriever = vectorstore.as_retriever()
document_chain = create_stuff_documents_chain(llm, prompt)
rag_chain = create_retrieval_chain(retriever, document_chain)

response = rag_chain.invoke({"input": "What is Nietzsche's idea of eternal recurrence?"})
```

It automates:

Step 1: Find relevant text → Step 2: Use LLM to answer from that.

Entire Code:

```
## RAG Q&A Conversation With PDF Including Chat History
import streamlit as st
from langchain.chains import create_history_aware_retriever, create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_chroma import Chroma
from langchain_community.chat_message_histories import ChatMessageHistory
from langchain_core.chat_history import BaseChatMessageHistory
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
```

```

from langchain_groq import ChatGroq
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import PyPDFLoader
import os

from dotenv import load_dotenv
load_dotenv()

os.environ['HF_TOKEN']=os.getenv("HF_TOKEN")
embeddings=HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

## set up Streamlit
st.title("Conversational RAG With PDF uploads and chat history")
st.write("Upload Pdf's and chat with their content")

## Input the Groq API Key
api_key=st.text_input("Enter your Groq API key:",type="password")

## Check if groq api key is provided
if api_key:
    llm=ChatGroq(groq_api_key=api_key,model_name="Gemma2-9b-It")

## chat interface

session_id=st.text_input("Session ID",value="default_session")
## statefully manage chat history

if 'store' not in st.session_state:
    st.session_state.store={}

uploaded_files=st.file_uploader("Choose A Pdf file",type="pdf",accept_multiple_files=True)
## Process uploaded PDF's

```

```

if uploaded_files:
    documents=[]
    for uploaded_file in uploaded_files:
        temppdf=f"./temp_{uploaded_file.name}"
        with open(temppdf,"wb") as file:
            file.write(uploaded_file.getvalue())
            file_name=uploaded_file.name

        loader=PyPDFLoader(temppdf)
        docs=loader.load()
        documents.extend(docs)

# Split and create embeddings for the documents
text_splitter = RecursiveCharacterTextSplitter(chunk_size=5000, chunk_o
verlap=500)
splits = text_splitter.split_documents(documents)
vectorstore = Chroma.from_documents(documents=splits, embedding=e
mbeddings)
retriever = vectorstore.as_retriever()

contextualize_q_system_prompt=(
    """
    Given a chat history and the latest user question which might reference
    context in the chat history, formulate a standalone question which can be und
    erstood without the chat history. Do NOT answer the question, just reformulat
    e it if needed and otherwise return it as is.
    """
)
contextualize_q_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", contextualize_q_system_prompt),
        MessagesPlaceholder("chat_history"),
        ("human", "{input}"),
    ]
)

```

```
history_aware_retriever=create_history_aware_retriever(llm,retriever,contextualize_q_prompt)
```

```
## Answer question
```

```
# Answer question
```

```
system_prompt = (
    "You are an assistant for question-answering tasks. "
    "Use the following pieces of retrieved context to answer "
    "the question. If you don't know the answer, say that you "
    "don't know. Use three sentences maximum and keep the "
    "answer concise."
    "\n\n"
    "{context}"
)
```

```
qa_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system_prompt),
        MessagesPlaceholder("chat_history"),
        ("human", "{input}"),
    ]
)
```

```
question_answer_chain=create_stuff_documents_chain(llm,qa_prompt)
```

```
rag_chain=create_retrieval_chain(history_aware_retriever,question_answer_chain)
```

```
def get_session_history(session:str)→BaseChatMessageHistory:
    if session_id not in st.session_state.store:
        st.session_state.store[session_id]=ChatMessageHistory()
    return st.session_state.store[session_id]
```

```
conversational_rag_chain=RunnableWithMessageHistory(
    rag_chain,get_session_history,
    input_messages_key="input",
    history_messages_key="chat_history",
)
```

```

        output_messages_key="answer"
    )

    user_input = st.text_input("Your question:")
    if user_input:
        session_history=get_session_history(session_id)
        response = conversational_rag_chain.invoke(
            {"input": user_input},
            config={
                "configurable": {"session_id":session_id}
            }, # constructs a key "abc123" in `store`.
        )
        st.write(st.session_state.store)
        st.write("Assistant:", response['answer'])
        st.write("Chat History:", session_history.messages)
    else:
        st.warning("Please enter the GRoq API Key")

```